20 LEETCODE PATTERNS CHEATSHEET

This comprehensive guide covers the 20 fundamental coding patterns required to ace LEETCODE problems. Mastering these patterns allows you to identify the structure of unseen problems, rather than memorizing thousands of unique solutions. Below, you will find each pattern, a brief explanation of its use case, and interactive links to problems where you need that pattern knowledge.

Pattern Breakdown and Curated Problems

Pattern	Description & Use Case	Interactive Problem Links
1. Sliding Window	Calculates metrics over a continuous subset of data (like an array or string). It dynamically adjusts the start and end of the "window" to satisfy a condition, reducing time complexity from O(N^2) to O(N).	• Maximum Subarray • Sliding Window Maximum • Minimum Size Subarray Sum • Sliding Window Median
2. Two Pointers	Utilizes two markers moving through a data structure (often sorted) from different directions or at different speeds to find pairs or meet a target sum.	• Valid Palindrome • Two Sum II - Input Array Is Sorted • 3Sum
3. Fast & Slow Pointers	Known as the Tortoise and Hare algorithm. It uses two pointers moving at different speeds to detect cycles in linked lists or find the middle element.	• Linked List Cycle • Linked List Cycle II • Happy Number
4. In-place Reversal of a	Modifies the nodes of a linked list to reverse their connections	• Reverse Linked List • Reverse Linked List II

Pattern	Description & Use Case	Interactive Problem Links
LinkedList	without using extra memory space.	
5. Merge Intervals	Deals with ranges or scheduling by sorting the start times and combining any intervals that overlap.	• Merge Intervals • Insert Interval
6. Cyclic Sort	A highly efficient sorting technique O(N) for arrays containing elements specifically in a continuous range (permutation like 1 to N).	• Missing Number
7. Modified Binary Search	Adapts standard O(log N) binary search for edge cases like rotated sorted arrays or infinite lists.	• Binary Search • Search in Rotated Sorted Array • Search in a Binary Search Tree
8. Tree Breadth-First Search (BFS)	Explores a tree structure level by level horizontally before moving deeper. Always uses a Queue.	• Binary Tree Level Order Traversal • Binary Tree Level Order Traversal II
9. Tree Depth-First Search (DFS)	Explores tree structures by going as deep as possible along each branch before backtracking. Uses Recursion or a Stack.	• Maximum Depth of Binary Tree • Path Sum
10. Topological Sort	Sorts vertices in a Directed Acyclic Graph (DAG) such that	• Course Schedule • Alien Dictionary

Pattern	Description & Use Case	Interactive Problem Links
	for every directed edge U to V, U comes before V. Used for scheduling.	
11. Union Find (Disjoint Set)	Efficiently tracks groups of connected elements in a graph and quickly detects if two elements are in the same group or form a cycle.	• Number of Islands • Redundant connections
12. Top 'K' Elements	Finds the extreme (largest, smallest, most frequent) K elements in an array using a Heap (Priority Queue).	• Top K Frequent Elements
13. Two Heaps	Uses a Max-Heap for the smaller half of numbers and a Min-Heap for the larger half to continuously track the median in a stream.	• Find Median from Data Stream
14. K-way Merge	Merges K number of sorted arrays or linked lists efficiently using a Min-Heap.	• Merge k Sorted Lists
15. Monotonic Stack	Maintains stack elements in an increasing or decreasing order. Essential for "Next Greater Element" type queries.Very large problemset available on this.	• Daily Temperatures • Largest Rectangle in Histogram

Pattern	Description & Use Case	Interactive Problem Links
16. Subsets (Backtracking)	Generates all combinations or permutations by exploring possibilities and abandoning those that fail conditions.	• Subsets • Permutations
17. 0/1 Knapsack (Dynamic Programming)	Optimization problems where you evaluate either taking an item or leaving it to achieve the maximum/minimum target value.	• Partition Equal Subset Sum
18. DP	Breaks down a main problem into smaller, overlapping sub-problems, storing their results to avoid redundant calculations.	• Climbing Stairs • House Robber
19. Trie (Prefix Tree)	A specialized tree used to store associative data structures. Crucial for autocomplete, spellcheck, and prefix searching.	• Implement Trie (Prefix Tree) • Word Search II
20. Bitwise XOR	Utilizes the XOR properties to find unique or missing elements in O(1) space.	• Single Number

Prepared by-
Mahmodur Rahman Sajib
BUET CSE'24

Feel free to pass any feedback regarding the patterns.